



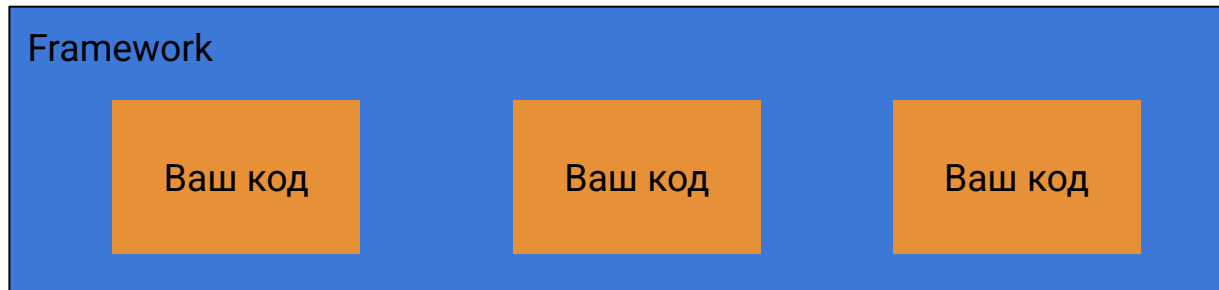
Лекція 6

Spring Framework. Spring Boot. Rest.

Поняття Framework

Фреймворк — програмна платформа, що визначає структуру програмної системи; програмне забезпечення, що полегшує розробку і об'єднання різних компонентів великого програмного проекту.

Wikipedia



Spring Framework Features

Core technologies: dependency injection, events, resources, i18n, validation, data binding, type conversion, SpEL, AOP

Data Access: transactions, DAO support, JDBC, ORM, Marshalling XML

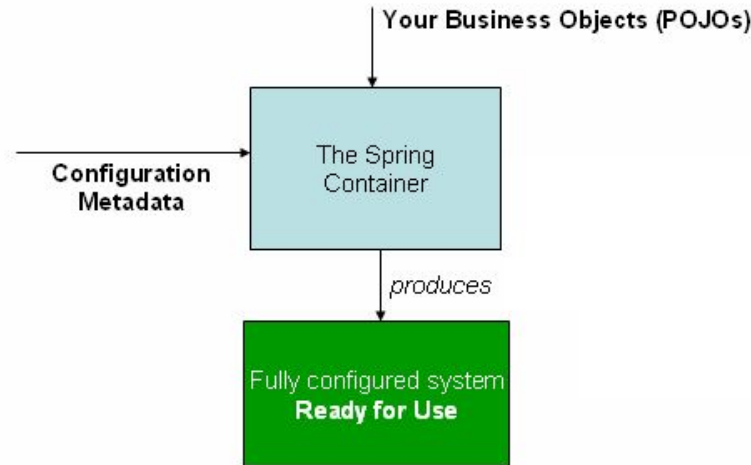
Spring MVC and **Spring WebFlux** web frameworks

Integration: remoting, JMS, JCA, JMX, email, tasks, scheduling, cache.

Testing: mock objects, TestContext framework, Spring MVC Test, WebTestClient

Container в Spring

Spring IoC Container створює об'єкти, налаштовує і пов'язує їх



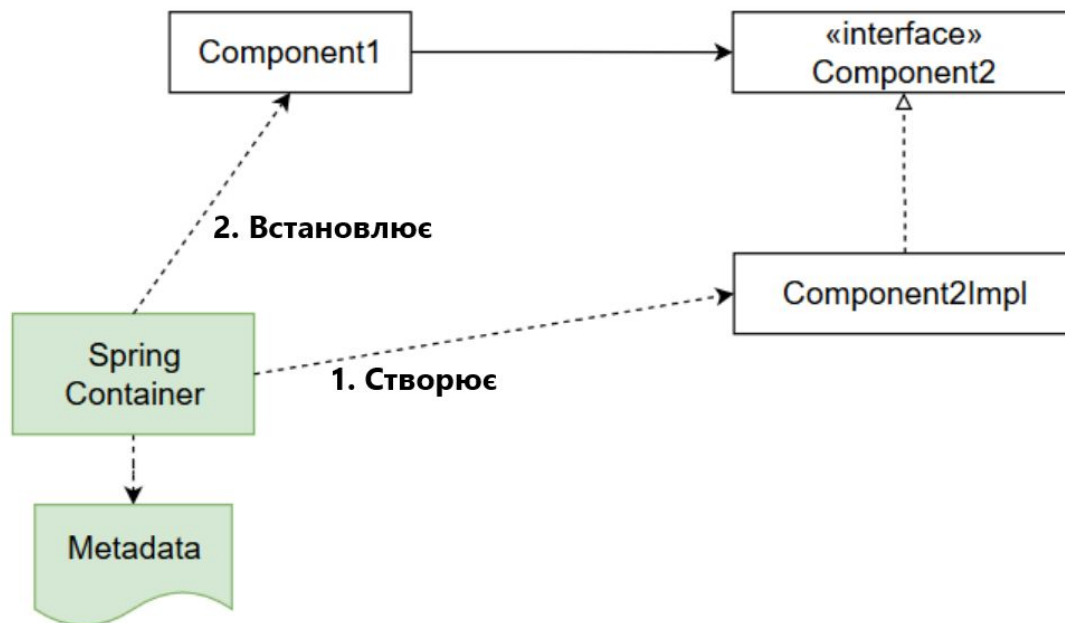
Dependency Injection (DI)

DI – це механізм передачі класу його залежностей

- через конструктор (Constructor Injection)
- через метод (Method Injection)
- через властивість (Property Injection)

```
public class StudentDao {  
  
    private final DataSource dataSource;  
  
    // Constructor Injection  
    public StudentDao(DataSource dataSource) {  
        this.dataSource = dataSource;  
    }  
  
    public Student getById(long id) { .. }  
}
```

DI в Spring



Configuration Metadata

Метадані конфігурації - це інструкції, за допомогою яких розробник повідомляє контейнеру, як створювати, налаштовувати і компонувати об'єкти всередині нього

Формати метаданих

- **XML**
 - `<bean>`, etc.
- **Annotation-based**
 - `@Component`, `@Autowired`, etc.
- **Java-based**
 - `@Configuration`, `@Bean`, etc.

XML-конфигурация

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="myBean" class="mypackage.MyBean">
        <!-- встановлення властивостей і залежностей -->
    </bean>

    <bean id="..." class="...">
        <!-- встановлення властивостей і залежностей -->
    </bean>

    <!-- ... -->

</beans>
```

Програмне створення контексту

// Створюємо контекст

ApplicationContext context =

```
    new ClassPathXmlApplicationContext(  
        "spring-context.xml");
```

// отримуємо бін із контексту

MyBean myBean =

```
    context.getBean("myBean", MyBean.class);
```

В *pom.xml* додаємо залежність

```
<dependency>
```

```
    <groupId>org.springframework</groupId>
```

```
    <artifactId>spring-context</artifactId>
```

```
    <version>5.2.3.RELEASE</version>
```

```
</dependency>
```

Конфігурація біна в XML - конструкторі

```
<beans>
```

```
<bean id="mssqlDataSource" class="..." />
```

```
<bean id="studentDao" class="ua.profitsoft.ajd.StudentDao">
```

```
<constructor-arg ref="mssqlDataSource"/>
```

```
</bean>
```

```
</beans>
```

Конфігурація біна в XML

Встановлення властивостей

```
<bean id="studentService"
      class="ua.profitsoft.ajd.StudentServiceImpl">
  <property name="dao" ref="studentDao" />
  <property name="maxReturnCount" value="20" />
  <property name="propertiesToDisplay">
    <list>
      <value>name</value>
      <value>group</value>
    </list>
  </property>
</bean>
```

Конфігурація біна в XML

Встановлення властивостей за допомогою p:

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:p="http://www.springframework.org/schema/p"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="studentService" class="ua.profitsoft.ajd.StudentServiceImpl"
        p:dao-ref="studentDao"
        p:maxReturnCount="10"
        p:propetiesToDisplay="name, value">

    </bean>

</beans>
```

Autowiring

Autowiring – це спосіб автоматичного встановлення залежностей між об'єктами

```
<bean id="studentService"
      class="ua.profitsoft.ajd.StudentServiceImpl"
      autowire="byType">
    <!-- посилання на StudentDao
           буде встановлене автоматично -->
</bean>
```

Наслідування визначень бінів

Конфігурації бінів можуть вибудовуватись в ієрархії

```
<bean id="bean1" abstract="true">  
  <!-- загальні властивості -->  
  <property name="commonProp" value="1"/>  
</bean>
```

```
<bean id="bean2" parent="bean1" class="...">  
  <!-- специфічні властивості -->  
  <property name="specProp" value="2"/>  
</bean>
```

Annotation-driven configuration

```
<?xml version="1.0" encoding="UTF-8"?>
<beans ...>
    <context:component-scan base-package="ua.profitsoft.jfd"/>
</beans>
```

@Repository

```
public class StudentDao {
    public StudentDao(DataSource dataSource) {
        this.dataSource = dataSource; }
    ...
}
```

@Service("studentService")

```
public class StudentServiceImpl implements StudentService {
```

@Autowired

```
private StudentDao dao;
```

@Value("\${studentService.maxReturnCount:20}")

```
private int maxReturnCount;
```

Java-based configuration

@Configuration

```
public class ApplicationConfig {
```

@Bean

```
public DataSource dataSource() {  
    HikariConfig config = new HikariConfig();  
    config.setDataSourceClassName(  
        "com.microsoft.sqlserver.jdbc.SQLServerDataSource");  
    ...  
    return new HikariDataSource(config);  
}  
}
```

Bean scopes

singleton - створюється один роздільний екземпляр об'єкта

- режим за замовчуванням

prototype - кожен раз буде створюватись незалежний об'єкт

request – об'єкт на рівні HTTP-запита

- Тільки для контексту, що знає про Web

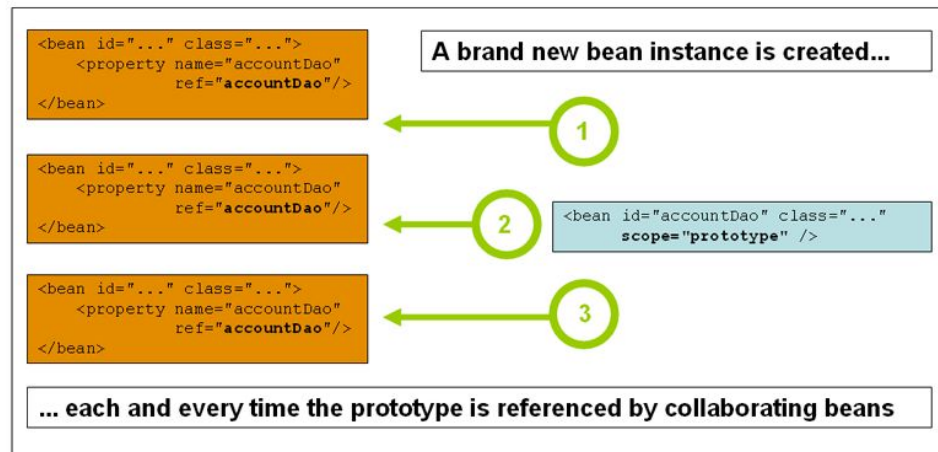
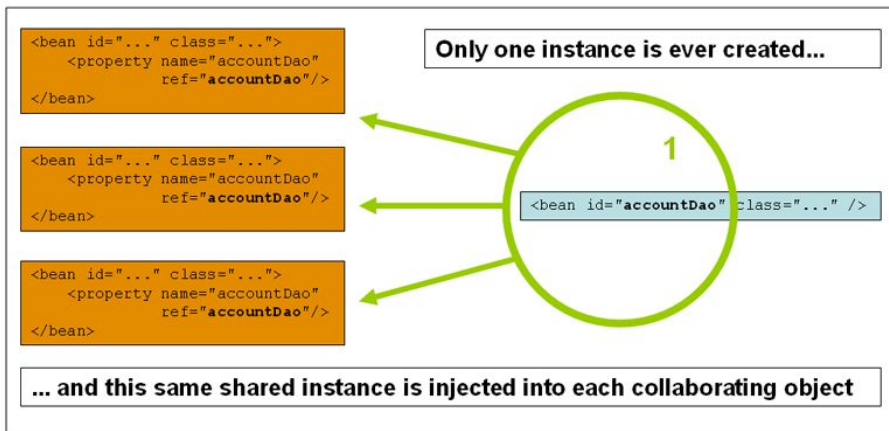
session – на рівні сесії

- Тільки для контексту, що знає про Web

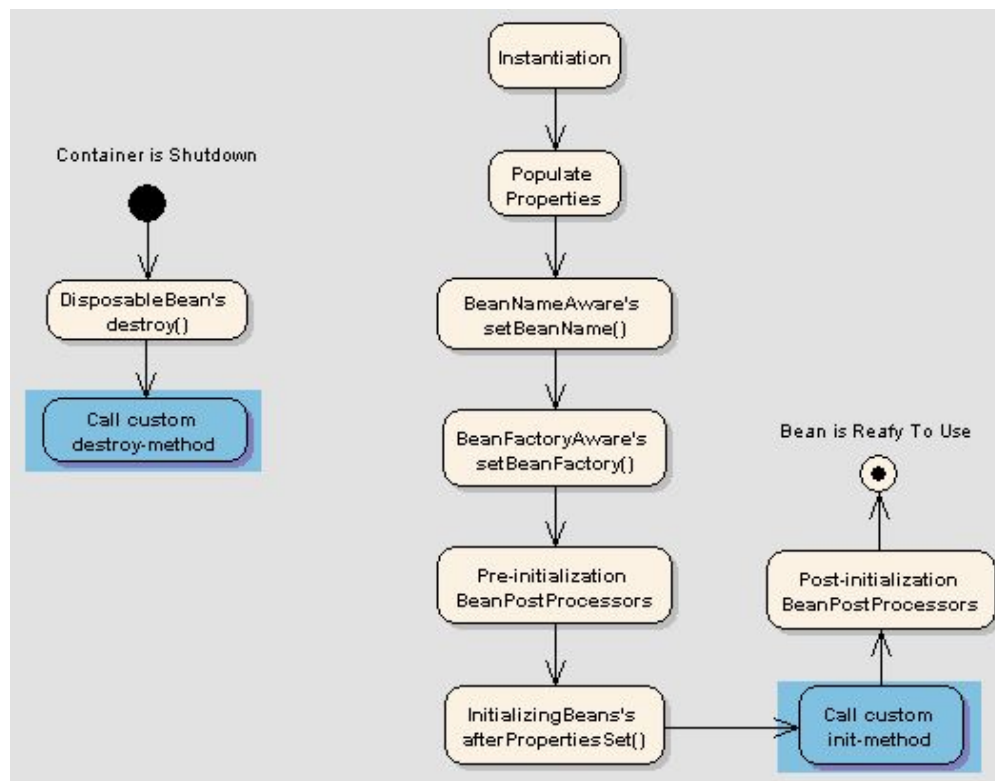
Спосіб задання

```
<bean id="..." class="..." scope="..." />  
або  
@Scope("prototype")
```

Singleton vs prototype



Життєвий цикл Spring Bean



@PostConstruct

@Service

```
public class StudentServiceImpl  
    implements StudentService {
```

@PostConstruct

```
public void init() {  
    // do some state checks or initialization logic  
}  
}
```

Configuration Properties в XML

```
<context:annotation-config/>
```

```
<context:property-placeholder
```

```
    location="classpath:jdbc.properties"/>
```

```
<bean class="org.springframework.jdbc.datasource.
```

```
    DriverManagerDataSource">
```

```
    <property name="url" value="${jdbc.url}"/>
```

```
    <property name="username" value="${jdbc.username}"/>
```

```
    <property name="password" value="${jdbc.password}"/>
```

```
</bean>
```

Configuration Properties з використанням анотацій

```
@Configuration
@ComponentScan(basePackages = "ua.profitsoft")
@PropertySource("classpath:app.properties")
public class ApplicationConfig {
    ...
}

@Service("studentService")
public class StudentServiceImpl implements StudentService {

    @Value("${studentService.maxReturnCount:20}")
    private int maxReturnCount;

    ...
}
```

Spring Boot

Spring Boot makes it easy to create stand-alone, production-grade Spring based Applications that you can "just run".

- Create stand-alone Spring applications
- **Embed Tomcat**, Jetty or Undertow directly (no need to deploy WAR files)
- Provide opinionated 'starter' dependencies to simplify your build configuration
- Automatically configure Spring and 3rd party libraries whenever possible
- Provide production-ready features such as metrics, health checks and externalized configuration
- Absolutely no code generation and no requirement for XML configuration



CM. <https://start.spring.io/>

Що таке REST

REST (Representational State Transfer) — архітектурний стиль взаємодії компонентів розподіленої інформаційної системи в мережі.

- Був описаний і популяризований 2000 року Роєм Філдінгом, одним із творців протоколу HTTP.

Ресурс — це ключовий елемент даних в REST. Ресурсом може бути що завгодно, що можна назвати:

- якийсь документ
- щось з реального світу (наприклад, працівник компанії)
- динамічне значення (наприклад, погода у Львові)

Для того, щоб посилатись на ресурси, використовуються **ідентифікатори ресурсів**

Особливості (обмеження) REST

- Клієнт-серверна архітектура
- Відсутність стану
- Різні формати даних
- Однорідний інтерфейс
- Можливість кешування

Приклади REST-запитів (ч.1)

Створення запису про студента

POST /students

```
{  
  "name": "Oleksandr",  
  ...  
}
```

Отримання даних студента

GET /students/{id}

Оновлення даних студента

PUT /students/{id}

```
{...}
```

Видалення даних студента

DELETE /students/{id}

Приклади REST-запитів (ч.2)

Отримання переліку студентів

(для великих масивів даних потрібні ще параметри)

GET /students

Пошук студентів*

POST /students/_search

```
{  
  "name": "Oleksandr",  
  "from": 21,  
  "size": 20  
}
```

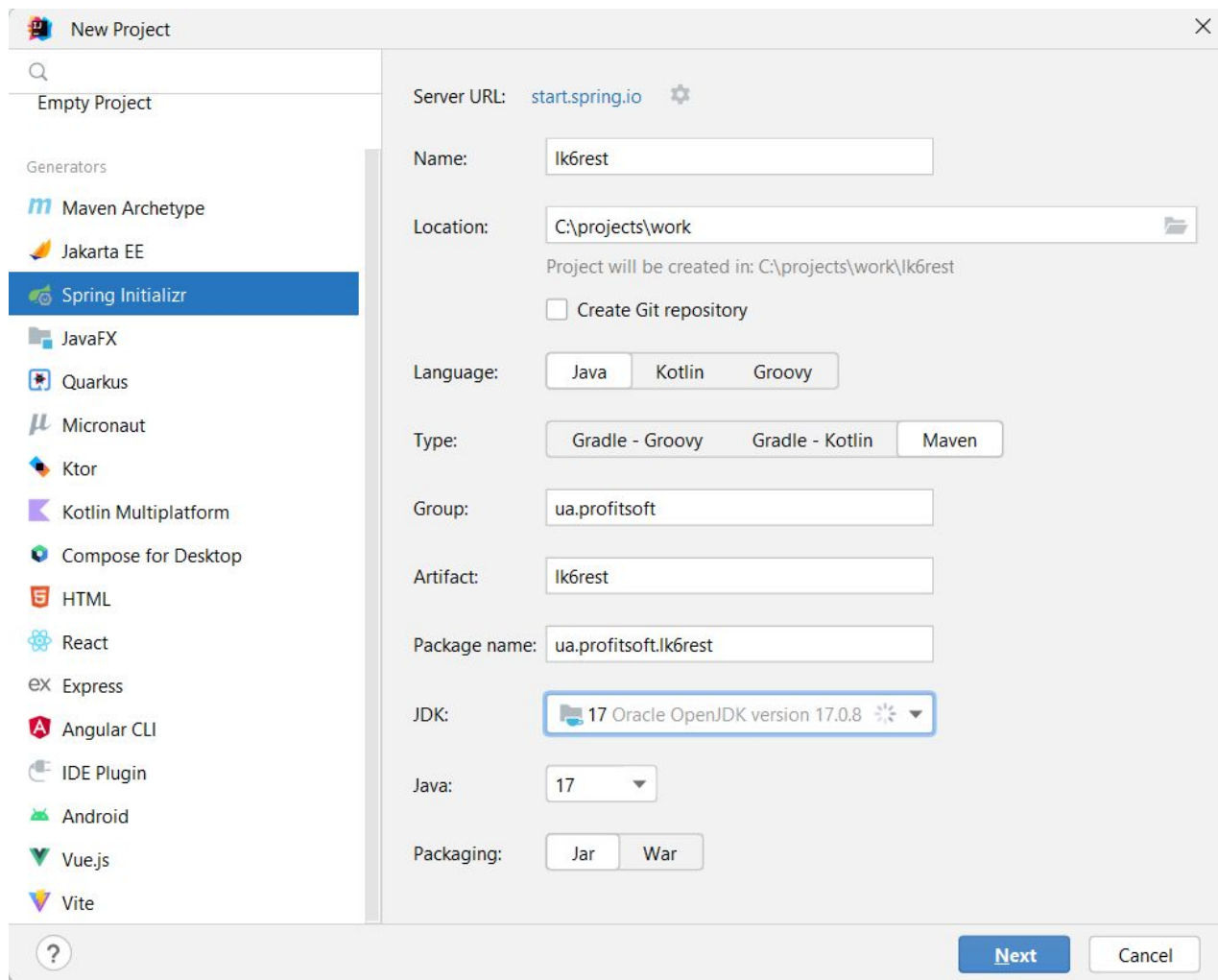
Випустити студента

POST /students/{id}/graduate

{...}

* Для POST-запитів, які насправді не модифікують дані, рекомендовано додавати префікс “_”.

New Spring Boot Project



New Project

Empty Project

Generators

- Maven Archetype
- Jakarta EE
- Spring Initializr**
- JavaFX
- Quarkus
- Micronaut
- Ktor
- Kotlin Multiplatform
- Compose for Desktop
- HTML
- React
- Express
- Angular CLI
- IDE Plugin
- Android
- Vue.js
- Vite

Server URL: start.spring.io

Name:

Location:

Project will be created in: C:\projects\work\lk6rest

☐ Create Git repository

Language: ☐ Java ☐ Kotlin ☐ Groovy

Type: ☐ Gradle - Groovy ☐ Gradle - Kotlin ☐ Maven

Group:

Artifact:

Package name:

JDK:

Java:

Packaging: ☐ Jar ☐ War

Next Cancel

New Spring Boot Project

New Project

Spring Boot:

Dependencies:

Search

- Developer Tools
 - ☐ GraalVM Native Support
 - ☐ GraphQL DGS Code Generation
 - ☐ Spring Boot DevTools
 - ☒ Lombok
 - ☐ Spring Configuration Processor
 - ☐ Docker Compose Support
 - ☐ Spring Modulith
- Web
 - ☒ Spring Web
 - ☐ Spring Reactive Web
 - ☐ Spring for GraphQL
 - ☐ Rest Repositories
 - ☐ Spring Session
 - ☐ Rest Repositories HAL Explorer
 - ☐ Spring HATEOAS
 - ☐ Spring Web Services
 - ☐ Jersey

Lombok
Java annotation library which helps to reduce boilerplate code.

Added dependencies:

- × Spring Web
- × Lombok

? Previous **Create** Cancel

StudentController (4.1)

```
@RestController
@RequestMapping("/api/students")
@RequiredArgsConstructor
public class StudentController {

    private final StudentService studentService;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public RestResponse createStudent(@RequestBody StudentSaveDto dto) {
        int id = studentService.createStudent(dto);
        return new RestResponse(String.valueOf(id));
    }
    ...
}
```

StudentController (4.2)

```
@GetMapping("/{id}")
```

```
public StudentDetailsDto getStudent(@PathVariable int id) {  
    return studentService.getStudent(id);  
}
```

```
@PutMapping("/{id}")
```

```
public RestResponse updateStudent(  
    @PathVariable int id,  
    @RequestBody StudentSaveDto dto  
) {  
    studentService.updateStudent(id, dto);  
    return new RestResponse("OK");  
}
```

```
@PostMapping("_search")
```

```
public List<StudentInfoDto> search(@RequestBody StudentQueryDto query) {  
    return studentService.search(query);  
}
```

Використовуюмо Postman

The screenshot displays the Postman web interface for a REST client. At the top, the URL bar shows `http://localhost:8080/api/students`. Below it, the **POST** method is selected from a dropdown menu. The **Body** tab is active, and the **raw** radio button is selected, with a **JSON** dropdown menu open. The request body contains a JSON object:

```
1 {  
2   "name": "Ivan",  
3   "surname": "Franko",  
4   "birthDate": "1856-08-27",  
5   "groupId": 2  
6 }
```

At the bottom, the **Body** tab is selected in the response section, showing the status **201 Created**, a time of **158 ms**, and a size of **272 B**. The response body is displayed in the **Pretty** view:

```
1 {  
2   "result": "1"  
3 }
```

<https://www.postman.com/downloads/>

Multipart and File Upload

```
@RestController
@RequestMapping("/api/students")
@RequiredArgsConstructor
public class StudentController {

    private final StudentService studentService;

    @PostMapping("/file/upload")
    @ResponseStatus(HttpStatus.CREATED)
    public RestResponse uploadFromFile(@RequestParam("file") MultipartFile multipart) {
        int count = studentService.uploadFromFile(multipart);
        return new RestResponse(count + " students uploaded from file");
    }
}
```

File Upload Via Postman

The screenshot displays the Postman interface for configuring a POST request. The URL bar shows `http://localhost:8080/api/students/file/upload`. The **POST** method is selected. The **Body** tab is active, showing the **form-data** type selected. A key-value pair is defined with the key **file** (checked) and the value **File** (selected) pointing to **pep.json**. The status bar at the bottom indicates the request was successful with a **Status: 201 Created**. The response body is shown in JSON format.

POST `http://localhost:8080/api/students/file/upload` **Send**

Params Authorization Headers (9) **Body** Pre-request Script Tests Settings Cookies

☐ none ☒ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

Key	Value	Description
☒ file	File <code>pep.json</code>	
Key	Text Value	Description

Body Cookies Headers (5) Test Results **Status: 201 Created** Time: 133 ms Size: 211 B Save as example

Pretty Raw Preview Visualize JSON

```
1 {  
2   "result": "3 students uploaded from file"  
3 }
```

File Download(4.1)

```
@RestController
@RequestMapping("/api/students")
@RequiredArgsConstructor
public class StudentController {

    private final StudentService studentService;

    @PostMapping("/report", produces = MediaType.APPLICATION_OCTET_STREAM_VALUE)
    public void generateReport(HttpServletResponse response) {
        studentService.generateReport(response);
    }
}
```

File Download (4.2)

@Override

```
public void generateReport(HttpServletResponse response) {  
    response.setContentType(MediaType.APPLICATION_OCTET_STREAM_VALUE);  
    response.setHeader(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=students.txt");  
    try {  
        StringBuilder sb = new StringBuilder();  
        List<StudentData> students = studentRepository.getAll();  
        for (StudentData student : students) {  
            sb.append(student.getName())  
                .append(" ")  
                .append(student.getGroup().getName())  
                .append("\n");  
        }  
        response.getOutputStream().write(sb.toString().getBytes());  
        response.getOutputStream().flush();  
    } catch (IOException e) {  
        throw new RuntimeException("Error generating text file", e);  
    }  
}
```

File Download Using Postman

The screenshot shows the Postman interface with a POST request to `http://localhost:8080/api/students/report`. The request is configured with the method **POST** and the URL `http://localhost:8080/api/students/report`. The **Body** tab is selected, and the message body is empty, displaying "This request does not have a body".

The response is a JSON array of student names, displayed in the **Body** tab. The response status is **200 OK**, with a time of **34 ms** and a size of **369 B**. The response is formatted as **Text**.

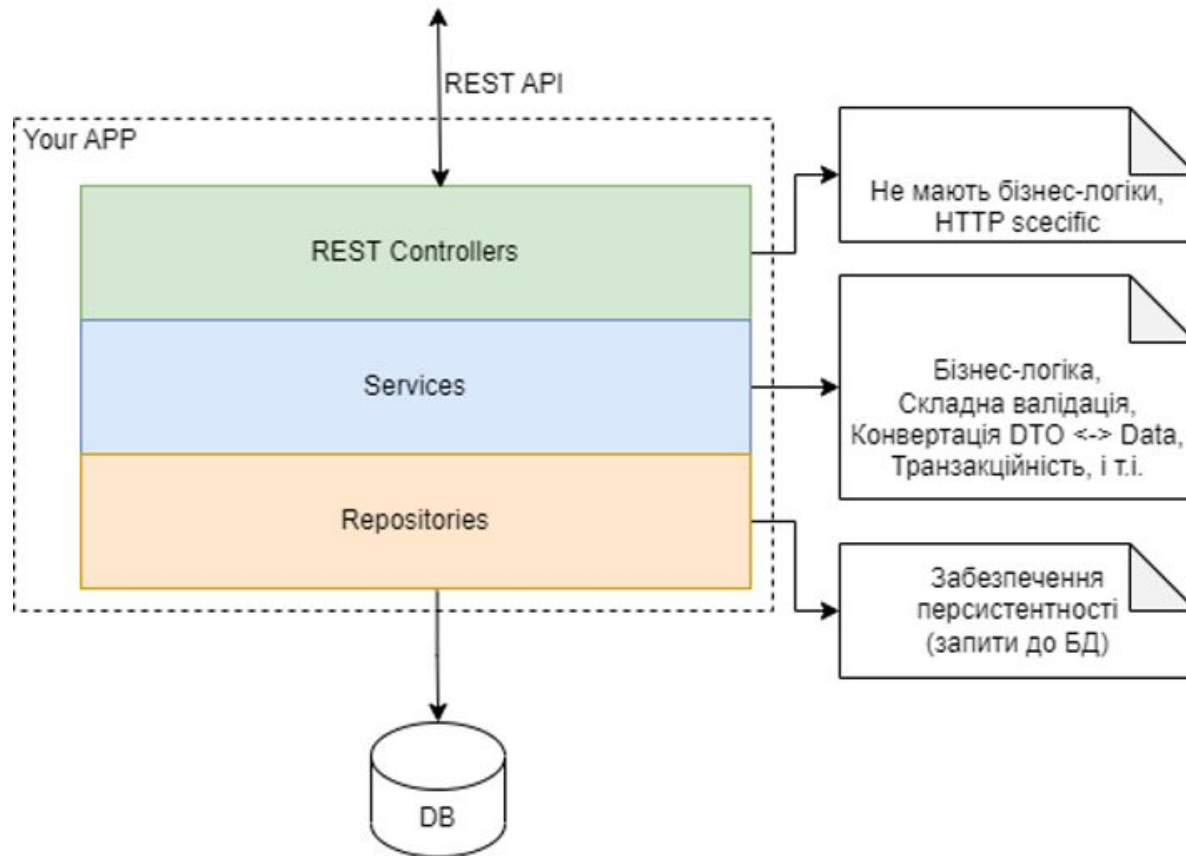
The response menu is open, showing the following options:

- Save response to file (highlighted)
- Clear response

The response data is as follows:

```
1 Тарас Group3
2 Леся Group2
3 Тарас Group1
4
```

Layered Application Design



Data Transfer Objects (DTOs)

// Коротка інформація для відображення в списках
StudentInfoDto

// Детальна інформація для відображення "картки студента"
StudentDetailsDto

// Інформація для створення/оновлення картки студента
StudentSaveDto

// Критерії пошуку студентів
StudentQueryDto

Validation

1. Додаємо залежність

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-validation</artifactId>  
</dependency>
```

2. Використовуємо анотації в DTO

```
@NotBlank(message = "name is required")  
private String name;
```

3. Задаємо анотацію @Valid в контролері

```
@PostMapping  
@ResponseStatus(HttpStatus.CREATED)  
public RestResponse createStudent(  
    @Valid @RequestBody StudentSaveDto dto) {  
    ...  
}
```

4. Додаємо property в application.properties

Обработка ошибок (@ResponseStatus)

```
@ResponseStatus(code = HttpStatus.NOT_FOUND)
public class NotFoundException extends RuntimeException {

    public NotFoundException(String message) {
        super(message);
    }
}
```

Обработка ошибок (@ExceptionHandler)

@Slf4j

@RestControllerAdvice

```
public class GlobalExceptionHandler {
```

```
    @ExceptionHandler(IllegalArgumentException.class)
```

```
    protected ResponseEntity<Object> handleIllegalArgumentException(IllegalArgumentException e) {
```

```
        log.warn("IllegalArgumentException thrown: {}", e.getMessage());
```

```
        return buildResponse(HttpStatus.BAD_REQUEST, e.getMessage());
```

```
    }
```

```
    private static ResponseEntity<Object> buildResponse(
```

```
        HttpStatus httpStatus, String message) {
```

```
        ErrorResponse response = new ErrorResponse(httpStatus.value(),
```

```
                                                    httpStatus.getReasonPhrase(), message);
```

```
        return ResponseEntity.status(httpStatus.value()).body(response);
```

```
    }
```

@Getter

@AllArgsConstructor

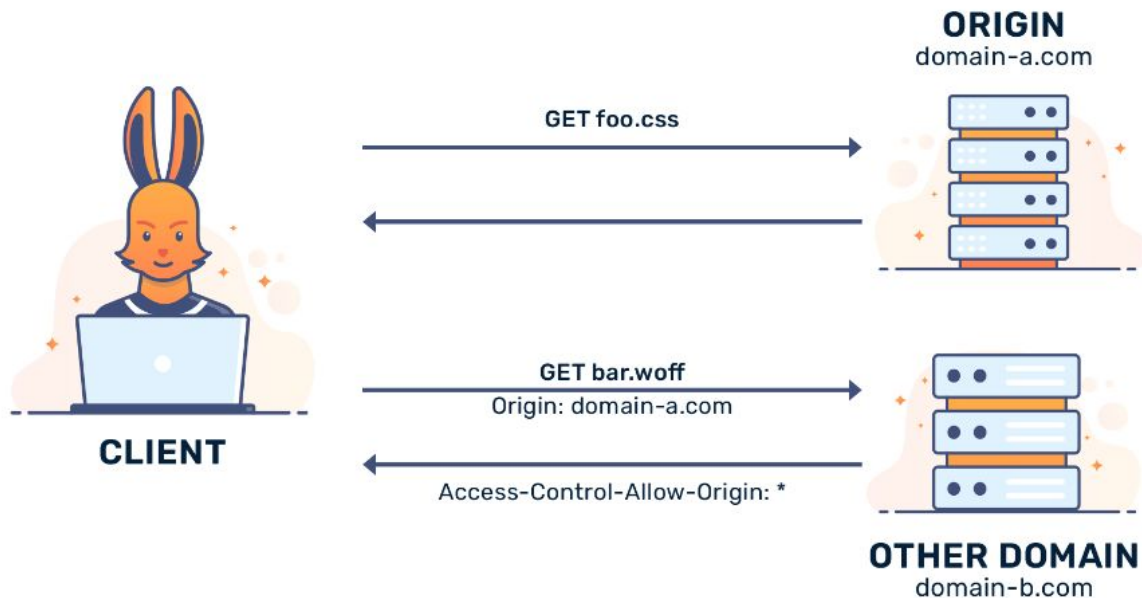
@JsonInclude(JsonInclude.Include.NON_NULL)

```
static class ErrorResponse {
```

```
    private int status;
```

CORS-Origins

CORS-заголовки – це спосіб, за допомогою якого сервери повідомляють браузерам, чи дозволено іншим доменам ділитися своїми ресурсами.



Enable CORS in Spring Boot

1) На рівні контролеру

```
@CrossOrigin(origins = "http://localhost:5500")
@RestController
@RequestMapping("/api/students")
@RequiredArgsConstructor
public class StudentController {

    private final StudentService studentService;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public RestResponse createStudent(@Valid @RequestBody StudentSaveDto dto) {
        int id = studentService.createStudent(dto);
        return new RestResponse(String.valueOf(id));
    }
}
```

2) На рівні методу

```
@CrossOrigin(origins = "http://localhost:5500")
@GetMapping("/{id}")
public StudentDetailsDto getStudent(@PathVariable int id) {
    return studentService.getStudent(id);
}
```

Global CORS Configuration

```
@Configuration
@EnableWebMvc
public class WebConfig {

    @Bean
    public CorsFilter corsFilter() {
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        CorsConfiguration config = new CorsConfiguration();
        config.addAllowedOrigin("http://localhost:9191");
        source.registerCorsConfiguration("/**", config);
        return new CorsFilter(source);
    }

    @Bean
    public FilterRegistrationBean<CorsFilter> corsFilterRegistrationBean() {
        FilterRegistrationBean<CorsFilter> bean = new FilterRegistrationBean<>(corsFilter());
        bean.setOrder(Ordered.HIGHEST_PRECEDENCE);
        return bean;
    }
}
```

Клієнтський застосунок

New Project

Spring Boot: 2.7.7

☒ Download pre-built shared indexes for JDK and Maven libraries

Dependencies:

- Search
- > Developer Tools
- > Web
- > Template Engines
- > Security
- > SQL
- > NoSQL
- > Messaging
- > I/O
- > Ops
- > Observability
- > Testing
- > Spring Cloud
- > Spring Cloud Tools
- > Spring Cloud Config
- > Spring Cloud Discovery
- > Spring Cloud Routing

Nothing selected

Added dependencies:

- × Lombok

? Previous Create Cancel

RestTemplate Config

@Configuration

```
public class RestTemplateConfig {
```

```
    public static final Duration REST_TEMPLATE_TIMEOUT = Duration.ofSeconds(30);
```

@Bean

```
    public RestTemplate restTemplate(RestTemplateBuilder restTemplateBuilder) {  
        return restTemplateBuilder  
            .setConnectTimeout(REST_TEMPLATE_TIMEOUT)  
            .setReadTimeout(REST_TEMPLATE_TIMEOUT)  
            .build();  
    }  
}
```

StudentsClient

@Component

@RequiredArgsConstructor

```
public class StudentsClient {  
    public static final String ENDPOINT_STUDENTS_CREATE = "/students";  
  
    private final RestTemplate restTemplate;  
  
    @Value("${studentsService.baseUrl:http://localhost:8080/api}")  
    private String studentsServiceBaseUrl;  
  
    public int createStudent(StudentSaveDto dto) {  
        ResponseEntity<RestResponse> response = restTemplate.exchange(  
            studentsServiceBaseUrl + ENDPOINT_STUDENTS_CREATE,  
            HttpMethod.POST,  
            new HttpEntity<>(dto),  
            ParameterizedTypeReference.forType(RestResponse.class)  
        );  
        String id = response.getBody().getResult();  
    }  
}
```

Calling a client

@Slf4j

@SpringBootApplication

```
public class Lk6clientApplication implements CommandLineRunner {
```

@Autowired

```
private StudentsClient studentsClient;
```

```
public static void main(String[] args) {
```

```
    SpringApplication.run(Lk9clientApplication.class, args);
```

```
}
```

@Override

```
public void run(String... args) {
```

```
    StudentSaveDto student = StudentSaveDto.builder()
```

```
        .name("Ivan")
```

```
        .surname("Test")
```

```
        .groupId(2)
```

```
        .build();
```

```
    int studentId = studentsClient.createStudent(student);
```

OkHttp Config

```
<dependency>  
  <groupId>com.squareup.okhttp3</groupId>  
  <artifactId>okhttp</artifactId>  
  <version>4.1.0</version>  
</dependency>
```

@Configuration

```
public class RestTemplateConfig {  
  
    public static final Duration REST_TEMPLATE_TIMEOUT = Duration.ofSeconds(30);  
  
    @Bean  
    public RestTemplate restTemplate(RestTemplateBuilder restTemplateBuilder) {  
        OkHttpClient client = new OkHttpClient.Builder()  
            .connectTimeout(REST_TEMPLATE_TIMEOUT)  
            .readTimeout(REST_TEMPLATE_TIMEOUT)  
            .build();  
  
        return restTemplateBuilder  
            .requestFactory(() -> new OkHttp3ClientHttpRequestFactory(client))  
    }  
}
```

HTTPS Server Configuration.

Self-Signed Certificate

Self-Signed Certificate - SSL-сертифікат, який користувач створив самостійно. Використовуються для тестування, або під час очікування на видачу офіційного сертифіката від авторитетного центру сертифікації.

Для генерації Self-Signed сертифіката потрібно використати таку команду в Terminal

```
keytool -genkeypair -alias example -keyalg RSA -keysize 4096 -storetype PKCS12 -keystore httpsKey.p12 -ext "SAN:c=DNS:localhost,IP:127.0.0.1" -validity 3650
```

- **genkeypair**: Генерує пару ключів
- **alias**: Псевдонім для сертифіката
- **ext**: Вказує, що сервер може бути знайдений на ім'я "localhost" та IP-адресу
- **keyalg**: Криптографічний алгоритм генерації ключів
- **keysize**: Розмір ключа
- **storetype**: Тип сховища ключів
- **keystore**: Назва сховища
- **validity**: Валідність ключа в днях
- **storepass**: Пароль до сховища

HTTPS Server Configuration.

SSL Properties

Щоб запустити `server` з `https` потрібно в `application.properties` додати такі параметри.

Тип сховища

```
server.ssl.key-store-type=PKCS12
```

Шлях до ключа

```
server.ssl.key-store=classpath:yourKey.p12
```

Пароль від ключа

```
server.ssl.key-store-password=123456
```

Alias обраний під час генерації

```
server.ssl.key-alias=ps-example
```

Вмикаємо SSL

```
server.ssl.enabled=true
```

HTTPS Client Configuration(ч.1)

Для того, щоб **Client** міг відправляти запити на сервер з HTTPS, потрібно підкласти наш сертифікат у RestTemplate. Спочатку додамо до **application.properties** нашого клієнта такі параметри.

Шлях до нашого ключа

```
trust.store=classpath:keystore/httpsKey.p12
```

Пароль від нашого ключа

```
trust.store.password=123456
```

HTTPS Client Configuration(4.2)

@Configuration

```
public class RestTemplateConfig {  
    public static final Duration REST_TEMPLATE_TIMEOUT = Duration.ofSeconds(30);
```

```
    @Value("${trust.store}")
```

```
    private Resource trustStore;
```

```
    @Value("${trust.store.password}")
```

```
    private String trustStorePass;
```

@Bean

```
public RestTemplate restTemplate(RestTemplateBuilder restTemplateBuilder) throws IOException,  
    CertificateException, NoSuchAlgorithmException, KeyStoreException, KeyManagementException {  
    SSLContext sslContext = new SSLContextBuilder()  
        .loadTrustMaterial(trustStore.getURL(), trustStorePass.toCharArray()).build();  
    CloseableHttpClient httpClient = HttpClients.custom()  
        .setSSLContext(sslContext)  
        .build();  
    ClientHttpRequestFactory requestFactory = new HttpComponentsClientHttpRequestFactory(httpClient);  
    restTemplateBuilder.requestFactory(() -> requestFactory)
```

Spring Profiles

Spring Profiles надають зручний спосіб відокремити частини конфігурації програми та зробити їх доступними лише в певних середовищах. Для того щоб обмежити щось для конкретного середовища, використовується анотація `@Profile("profName")`.

1. На рівні методу

```
@Bean(name = "restTemplate")  
@Profile("dev")  
public RestTemplate devTemplate(RestTemplateBuilder restTemplateBuilder)
```

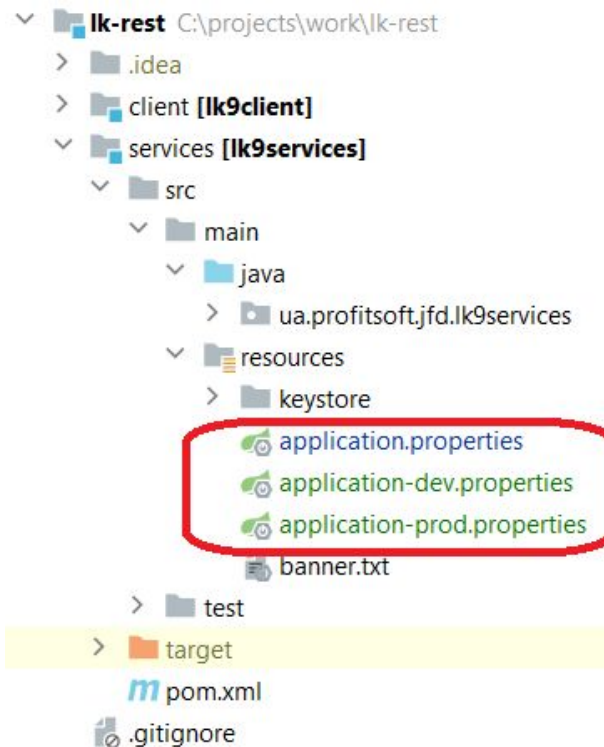
2. На рівні класу

```
@Service  
@Profile("dev")  
public class DevService {}
```

Якщо профіль `dev` не обраний, то ні об'єкт класу `DevService`, ні `RestTemplate` не буде створено.

Profile-specific Properties Files

Profile-specific Properties Files - дозволяють визначати параметри конфігурації в залежності від активного профілю. Назва файлу повинна бути в форматі *application-{profile}.properties* та знаходитись у *resources* папці.



Spring Profiles Enabling

Run/Debug Configurations

Spring Boot

- Lk9clientApplication
- Lk9servicesApplication**

Name: Lk9servicesApplication ☐ Store as project file

Run on: Local machine [Manage targets...](#)

Run configurations may be executed locally or on a target; for example in a Docker Container or on a remote host using SSH.

Build and run [Modify options](#) Alt+M

java 17 SDK of 'lk9serv: -cp lk9services ua.profitsoft.jfd.lk9services.Lk9servicesApplication

Press Alt for field hints

Active profiles: **dev**

Comma separated list of profiles

☐ Open run/debug tool window when started ☐ Add dependencies with "provided" scope to classpath

[Edit configuration templates...](#)

Run OK Cancel Apply

Integration Tests

```
@SpringBootTest(  
    webEnvironment = SpringBootTest.WebEnvironment.MOCK,  
    classes = Lk8services1Application.class)
```

```
@AutoConfigureMockMvc
```

```
public class StudentControllerTest {
```

```
    @Autowired
```

```
    private MockMvc mvc;
```

```
    @Autowired
```

```
    private StudentRepository studentRepository;
```

```
@AfterEach
```

```
public void afterEach() {  
    studentRepository.deleteAll();  
}
```

```
@Test
```

Integration Test (4.2)

@Test

```
public void testCreateStudent() throws Exception {  
    String name = "Oleksandr";  
    String surname = "Rudenko";  
    int groupId = 2;  
    String body = ""  
        {  
            "name": "%s",  
            "surname": "%s",  
            "birthDate": "2001-12-12",  
            "groupId": %d  
        }  
        ""formatted(name, surname, groupId);  
  
    MvcResult mvcResult = mvc.perform(post("/api/students")  
        .contentType(MediaType.APPLICATION_JSON)  
        .content(body)  
    )
```

Integration Test (ч.3)

@Test

```
public void testCreateStudent() throws Exception {
```

```
    ... // дивись попередній слайд
```

```
    RestResponse response = parseResponse(mvcResult, RestResponse.class);
```

```
    int studentId = Integer.parseInt(response.getResult());
```

```
    assertThat(studentId).isGreaterThanOrEqualTo(1);
```

```
    StudentData student = studentRepository.get(studentId).orElse(null);
```

```
    assertThat(student).isNotNull();
```

```
    assertThat(student.getName()).isEqualTo(name);
```

```
    assertThat(student.getSurname()).isEqualTo(surname);
```

```
    assertThat(student.getGroup().getId()).isEqualTo(groupId);
```

```
}
```

Test Validation

@Test

```
public void testCreateStudent_validation() throws Exception {  
    mvc.perform(post("/api/students")  
        .contentType(MediaType.APPLICATION_JSON)  
        .content("{}")  
    )  
    .andExpect(status().isBadRequest());  
}
```